

CSAPP
期末复习

第一章 计算机系统漫游

- 编译过程分为四个阶段：预处理、编译、汇编和链接。要记住比如hello.c->hello.i->hello.s->hello.o->可执行目标文件，这个过程转换过程分别是由上述哪个阶段完成。
参考教材P3 图1-3。
- 计算机系统抽象概念的抽象概念：见教材P19-P20
- 1) 文件是对 I/O 设备的抽象表示。
- 2) 虚拟内存是对主存和磁盘 I/O 设备的抽象表示。
- 3) 进程则是对处理器、主存和 I/O 设备的抽象表示。
- 4) 在处理器里，指令集架构提供了对实际处理器硬件的抽象。
- 5) 虚拟机，它提供对整个计算机的抽象，包括操作系统、处理器和程序。
- 虚拟内存中的堆和栈
- 1) 堆：代码和数据区后紧接着的是运行时堆。当调用像 malloc 和 free 这样的 C 标准库函数时，堆可以在运行时动态地扩展和收缩。
- 2) 栈：用户栈在程序执行期间可以动态地扩展和收缩。特别地，每次我们调用一个函数时，栈就会增长；从一个函数返回时，栈就会收缩。

第二章 信息的表示和处理

- 整数的机器码表示，默认就是补码，但要注意到底是多少位，比如int是32位长度
- 大端方式和小端方式下，数据中各个字节和内存地址的对应关系
- 对于大多数 C 语言的实现，处理同样字长的有符号数和无符号数之间相互转换的一般规则是：数值可能会改变，但是位模式不变。但处理不同长度的数之间的转换，要搞清楚截断及（有符号、无符号）扩展，以及扩展后的重新表达。
- 有符号数和无符号数的转换
- 当执行一个运算时，如果它的一个运算数是有符号的而另一个是无符号的，那么 C 语言会隐式地将有符号参数强制类型转换为无符号数，并假设这两个数都是非负的，来执行这个运算。
- 溢出
- 有符号数和无符号数的溢出判断准则及区别。什么是正溢出，什么是负溢出？对于CF和OF影响
- IEEE754
- 简答题：请结合 ieee754 编码，说明怎样判断两个浮点数是否相等？
- 答：由于浮点数的 ieee754 编码表示存在着精度、舍入、溢出、类型不匹配等问题，两个浮点数不能够直接比较大小，应计算两个浮点数的差的绝对值，当绝对值小于某个可以接受的数值（精度）时认为相等。
- 舍入
- 整数除法舍入：向零舍入（舍入到零）
- 浮点数舍入：当数字是中间值时，将数字向上或者向下舍入，使得结果的最小有效数字是偶数（即向偶舍入），否则直接是向上或者向下舍入。
- double、int、float之间的转换现象（溢出、精度损失、没影响等），必须记住
- 整数和浮点数 加法、乘法产生的各种特殊情况，包括大数吃小数现象，尤其是交换律和结合律有几个特殊的例子，查看PPT

第三章 程序的机器级表示

- 本章记忆内容都在速记清单里，包括不同寄存器的功能，还有X86常用的指令，以教材内容为主，速记清单所有内容都要记。
- 计算给定的结构体对齐所需的最小字节数，参考ppt的例题
- 简述缓冲区溢出攻击的原理以及防范方法（5分）。
答：攻击原理（3个采分点）：
(1) 程序输入缓冲区写入特定的数据，例如在gets读入字符串时
(2) 使位于栈中的缓冲区数据溢出，用特定的内容覆盖栈中的内容，例如函数返回地址等
(3) 使得程序在读入字符串，结束函数gets从栈中读取返回地址时，错误地返回到特定的位置，执行特定的代码，达到攻击的目的。
防范方法(2个采分点)：
(1) 代码中避免溢出漏洞：例如使用限制字符串长度的库函数。
(2) 随机栈偏移：程序启动后，在栈中分配随机数量的空间，将移动整个程序使用的栈空间地址。
(3) 限制可执行代码的区域
(4) 进行栈破坏检查—金丝雀
- 重要题型：汇编和C之间的互相转换填空，汇编填空或者读汇编写结果，参见作业题和PPT例题及本期期末试卷
- 重要题型：本章的汇编和第七章链接容易出大题，见最后一次作业的最后一大题。解释汇编代码以及填写重定位后的相对地址或绝对地址

第四章 处理器体系结构

- 流水线的计算，见作业题
- 重要题型：Y86-64微指令机器码格式及Y86-64微指令，给定指令，写出其对应的六个阶段的微指令，见作业题和历年试卷
- 基本的Y86电路图元素，能看懂会填空及分析。参考本部和深圳的历年试卷。

第五章 优化程序性能

- 必须能够写出几种常用的优化方法：
- 代码移动
- 复杂运算简化
- 共享公共子表达式
- 减少过程调用
- 消除不必要的内存引用
- 循环展开
- 重要题型：给定程序，能够从几个方面进行优化，同时说出所用的优化方法，见作业题类型及本部历年试卷，以及PPT上的三层for循环的优化方法例子
- 简答题：列举至少5种程序优化的方法。
答：代码移动，通过将代码从循环中移出减少计算执行的频率
用简单计算替代复杂计算/操作，如移位、加替代乘法/除法
共享共用子表达式、重用表达式的一部分
使用局部变量作为累积量
循环体展开减少循环次数
通过多个累积量、重新结合（组合）的方法，提高指令级并行性
尽量缩短关键路径（利于流水操作）
减少过程调用，小函数使用 inline 形式声明
用条件表达式替代 if-else 语句（用功能性的风格重写条件操作，有利于编译器
采用条件数据传送实现，而非使用分支结构实现）
消除不必要的内存引用
使用速度更快的 CPU 指令，例如 SSE 或 SIMD 指令。

第六章 存储器层次结构

- 局部性原理
- 定义：程序倾向于引用邻近于其他最近引用过的数据项的数据项，或者最近引用过的数据项本身。局部性通常有两种不同的形式：时间局部性和空间局部性，在一个具有良好时间局部性的程序中，被引用过一次的内存位置很可能在不远的将来再次被多次引用。在一个具有良好空间局部性的程序中，如果一个内存位置被引用了一次，那么程序很可能在不远的将来引用附近的一个内存位置。
- 优化：重复引用相同变量的程序有良好的时间局部性。对于具有步长为1的引用模式的程序，步长越小，空间局部性越好。对于取指令来说，循环有好的时间和空间局部性。循环体越小，循环迭代次数越多，局部性越好。
- 存储器层次结构，寄存器、cache、主存、磁盘，对比其速度、容量和成本，分别是怎样的排序
- 重要题型：直接映射、全相联、组相联相关计算
- cache与主存的数据交换单位是“块”；主存和辅存（磁盘）的数据交换单位是“页面”；TLB是对“页表”的缓存
Cache是对“主存/内存”的缓存。
- 计算：磁盘平均访问时间
- 写穿透和写回法的定义及理解
- 计算：不命中率
- 缓存不命中中的种类：
1) 冷不命中：如果第 k 层的缓存是空的（冷缓存），那么对任何数据对象的访问都会不命中。
2) 冲突不命中：限制性的放置策略引起的不命中。
3) 容量不命中：工作集的大小超过缓存的大小

第七章 链接

- 链接器的两个阶段：符号解析、重定位
- 符号：对应于一个函数、一个全局变量或一个静态变量（注意，这里是C语言中任何以static属性声明的变量）
- 强符号，弱符号在链接时候的三种规则，会判断
强符号：函数和初始化全局变量；弱符号：未初始化的全局变量
规则 1:不允许多个同名的强符号，每个强符号只能定义一次；否则，链接器错误
规则 2:若有一个强符号和多个弱符号同名，则选择强符号；对弱符号的引用解析为强符号
规则 3:如果有多个弱符号，选择任意一个
- 简答题：简述静态库及其优点。
答：编译时将所有相关的目标模块打包成为一个单独的文件用作链接器的输入，这个文件称为静态库。使用时，通过把相关函数编译为独立模块，然后封装成一个单独的静态库文件，应用程序可以通过在命令行上指定单独的文件名来使用这些在库中定义的函数，链接器只需复制被程序引用的目标模块，减少了可执行文件在磁盘和内存中的大小；同时，应用程序员只需包含较少的库文件的名称。
- 简答题：简述链接器在目标文件生成可执行文件时如何处理符号引用/解析的。
答：1) 全局符号中：有初值的全局变量、函数是强符号，无初值的全局变量是弱符号
2) 根据强符号类别，确定被引用的符号在哪里
3) 根据各目标文件的段落大小、系统代码等信息，合并同类型的段，并计算可执行文件各段的大小
4) 根据程序的内存映像，确定被引用的符号在程序运行时的内存地址
5) 根据在磁盘上的可执行文件中，各段落的大小，确定符号引用的位置
6) 根据上述信息，计算符号引用处应该采用的数值，并写入文件。
- 简答题：共享库（动态链接库）的定义及实现方法。
答：共享库（动态链接库）是一个so的目标模块（elf文件），在运行或加载时，由动态链接器程序加载到任意的内存地址，并和一个和内存中的程序（如当前可执行目标文件）动态完全连接为一个可执行程序。使用它可节省内存与硬盘空间，方便软件的更新升级。如标准C库libc.so。
动态链接的实现方法
加载时动态链接：应用程序第一次加载和运行时，通过ld-linux.so动态链接器重定位动态库的代码和数据到某个内存段，再重定位当前应用程序中对共享库定义的符号的引用，然后将控制传递给应用程序（此后共享库位置固定了并不变）。
运行时动态链接：在程序执行过程中，通过dlopen/dlsym函数加载和连接共享库，实现符号重定位，通过dlclose卸载动态库。
- 重要题型：结合汇编和机器码计算重定位相对地址和绝对地址
- 库打桩：允许截获对共享库函数的调用，取而代之执行自己的代码。具体分为以下三种：
1) 编译时:源代码被编译时
2) 链接时:当可重定位目标文件被静态链接来形成一个可执行目标文件时
3) 加载/运行时:当一个可执行目标文件被加载到内存中，动态链接，然后执行时
- .text节:已编译程序的机器码
.data节:已初始化的全局和静态C变量
.bss:未初始化的全局和静态C变量
有时候还这样进一步划分：
1) COMMON 未初始化的全局变量
2) .bss 未初始化的静态变量，以及初始化为0的全局或静态变量

第八章 异常控制流

- 异常的分类及具体区别，特别喜欢考缺页异常，见PPT
- 进程上下文包含哪些元素？（见教材511）
上下文就是内核重新启动一个被抢占的进程所需的状态。它由一些对象的值组成，这些对象包括通用目的寄存器、浮点寄存器、程序计数器、用户栈、状态寄存器、内核栈和各种内核数据结构，比如描述地址空间的页表、包含有关当前进程信息的进程表，以及包含进程已打开文件的信息的文件表。
- 常用信号量SIGXXX必须记忆，并能结合程序分析结果
- fork, signal, wait, waitpid等函数需要记忆，尤其结合具体程序分析
- 进程的状态：运行，停止或者终止
- fork、execve、setjmp、longjmp这几个函数分别是调用一次返回几次？
- 重要题型：根据程序画进程图，或者给定进程图并回答父进程和子进程分别输出什么，及对输出顺序的正确与否进行判断。
- 简答题：结合 fork、execve 函数，简述在 shell 中加载和运行 hello 程序的过程。
答：① 在 shell 命令中输入命令：\$./hello
② shell 命令解释器构造 argv 和 envp；
③ 调用 fork()函数创建子进程，其地址空间与 shell 父进程完全相同，包括只读代码段、读写数据段、堆及用户栈等
④ 调用 execve()函数在当前进程（新创建的子进程）的上下文中加载并运行hello 程序。将 hello 中的.text 节、.data 节、.bss 节等内容加载到当前进程的虚拟地址空间
⑤ 调用 hello 程序的 main()函数，hello 程序开始在一个进程的上下文中运行。
- 简答题：shell 的概念
答：Linux 系统中，Shell 是一个交互型应用级程序，代表用户运行其他程序(是命令解释器，以用户态方式运行的终端进程)。其基本功能是解释并运行用户的指令，重复如下处理过程：
(1)终端进程读取用户由键盘输入的命令行；
(2)分析命令行字符串，获取命令行参数，并构造传递给 execve 的 argv 向量；
(3)检查第一个(首个、第 0 个)命令行参数是否是一个内置的 shell 命令；
(4)如果不是内部命令，调用 fork()创建新进程/子进程；
(5)在子进程中，用步骤 2 获取的参数，调用 execve()执行指定程序；
(6)如果用户没要求后台运行(命令末尾没有&号)，则shell使用 waitpid (或 wait.)等待作业终止后返回；
(7)如果用户要求后台运行(如果命令末尾有&号)，则shell返回。

第九章 虚拟内存

- 结合图，简述虚拟内存地址翻译的过程。（见本章作业）
答：(1) 处理器将虚拟地址发送给 MMU
(2)、(3) MMU 利用虚拟地址对应的虚拟页号生成页表项（PTE）地址，并从页表中找到对应的 PTE
(4) PTE 中的有效位为 0，MMU 触发缺页异常
(5) 缺页处理程序选择物理内存中的牺牲页（若页面被修改，则换出到磁盘）
(6) 缺页处理程序调入新的页面到内存，并更新 PTE
(7) 缺页处理程序返回到原来进程，再次执行导致缺页的指令
- 空闲块的四种组织形式及优缺点，见PPT
- 动态内存分配的内存碎片产生原因：
1) 维护数据结构产生的开销
2) 增加块大小以满足对齐的约束条件
3) 显式的策略决定
- 虚拟内存缺页的定义及处理方式，属于（第八章提到的四种异常）哪类异常？
- 重要题型：给定core i7的一个存储结构图，写出VA、PA、VPN、VPO、TLBI、TLBT、CI、CT、CO或者对应位数，见作业题及历年期末试卷

第十章 系统级IO

- 不足值的产生原因
- 内核用三个相关的数据结构来表示打开的文件：
描述符表、打开文件表、vnode表
- IO重定向，dup2函数的正确使用
- Unix I/O、C标准I/O和RIO的特点和适用场景
- 阅读I/O程序并分析